LAB EXPERIMENT 9

Experiment 9: Rule-Based POS Tagging Using Regular Expressions

Aim

To implement a rule-based part-of-speech tagger using regular expressions to identify word patterns.

Algorithm

1. Start the program.
2. Import RegexpTagger from NLTK.
3. Define regular-expression patterns for common word endings.
4. Create a RegexpTagger using the patterns in order.
5. Tokenize a sample sentence.
6. Apply the tagger to the tokens and display the assigned tags.
7. Stop the program.

Code

```
from nltk.tag import RegexpTagger

patterns = [
    (r".*ing$", "VBG"),   # gerund or present participle
    (r".*ed$", "VBD"),    # past tense verb
    (r".*ly$", "RB"),     # adverb
    (r".*s$", "NNS"),     # plural noun
    (r".*", "NN")         # default: noun
]

tagger = RegexpTagger(patterns)
words = ["playing", "walked", "quickly", "cats", "book"]

print("Word -> POS Tag")
for word, tag in tagger.tag(words):
    print(word, "->", tag)
```

Output / Screenshot

```python
import re

patterns = [
    (r".*ing$", "VBG"),
    (r".*ed$", "VBD"),
    (r".*ly$", "RB"),
    (r".*s$", "NNS"),
    (r".*", "NN")
]

def pos_tag(word):
    for pattern, tag in patterns:
        if re.fullmatch(pattern, word):
            return tag

words = ["playing", "walked", "quickly", "cats", "book"]

for word in words:
    print(word, "->", pos_tag(word))
```

```
playing -> VBG
walked -> VBD
quickly -> RB
cats -> NNS
book -> NN
```

Result

Thus, a rule-based POS tagger was implemented using regular expressions to assign tags according to word patterns.